

CMU 10-703: Deep Reinforcement Learning

Lecture 3: Policy Gradients, REINFORCE, and Actor-Critic Methods

Max Simchowitz

Carnegie Mellon University

1.

Policy Optimization

CMU 10-703: Deep Reinforcement Learning

Direct Policy Optimization

Policy objective. Choose parameters θ that maximize expected return under the stochastic policy π_θ .

$$\max_{\theta} U(\theta), \quad \theta_{\text{new}} = \theta_{\text{old}} + \alpha \nabla_{\theta} U(\theta).$$

Continuous actions

Optimize a distribution over a high-dimensional action space.

Stochastic behavior

Represent uncertainty and exploration directly in the policy.

Direct objective

Improve the policy without first extracting it from a value function.

Policy gradients turn reinforcement learning into stochastic optimization over policy parameters.

Policy Optimization Loop

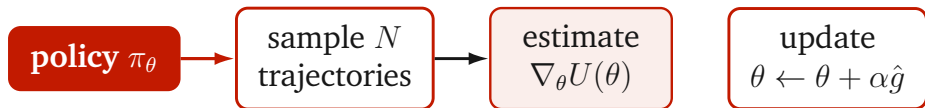


1. Collect

Deploy the current policy in the environment.

The policy determines which trajectory data are collected.

Policy Optimization Loop



1. Collect

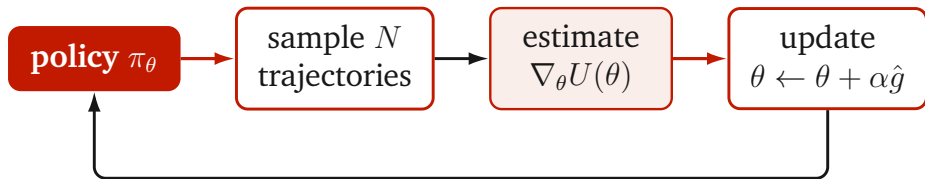
Deploy the current policy in the environment.

2. Estimate

Use sampled rewards to construct a gradient estimate.

The central problem is estimating a gradient from sampled trajectories.

Policy Optimization Loop



1. Collect

Deploy the current policy in the environment.

2. Estimate

Use sampled rewards to construct a gradient estimate.

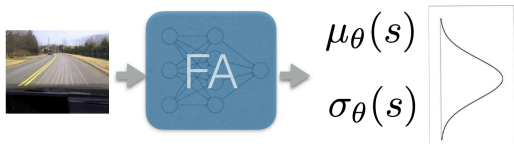
3. Improve

Move parameters in the estimated ascent direction.

Each update changes the distribution that supplies the next batch of data.

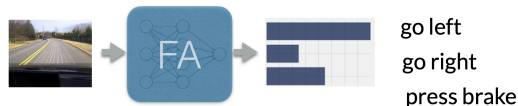
Stochastic Policy Classes

Continuous actions



A network outputs a Gaussian mean and covariance.

Discrete actions



A network outputs logits for a categorical distribution.

This lecture studies stochastic Gaussian and softmax policies, not deterministic policies.

Gaussian and Softmax Parameterizations

Gaussian policy

$$a \sim \mathcal{N}(\mu_{\theta}(s), \Sigma_{\theta}(s))$$

Common simplifications use diagonal covariance or a fixed variance.

Softmax policy

$$\pi_{\theta}(a \mid s) = \frac{\exp h_{\theta}(s, a)}{\sum_b \exp h_{\theta}(s, b)}$$

The score $h_{\theta}(s, a)$ can be linear in features or produced by a neural network.

A differentiable distribution converts network outputs into action probabilities.

Coordinate Finite Differences

Finite-difference gradient. Perturb one coordinate of a low-dimensional parameter vector and measure the objective change.

$$\frac{\partial U(\theta)}{\partial \theta_k} \approx \frac{U(\theta + \epsilon u_k) - U(\theta)}{\epsilon}, \quad k = 1, \dots, n.$$

Strength

Applies even when the policy or objective is not differentiable.

Cost

Requires one perturbation per coordinate, plus a shared evaluation of $U(\theta)$.

Finite differences are practical only when the policy has few parameters or evaluations are inexpensive.

AIBO Running Policy



Twelve policy parameters

1. Create positive, zero, and negative perturbations.
2. Run each perturbed gait on the physical course.
3. Compare traversal scores coordinate by coordinate.
4. Normalize the resulting direction and update the gait.

Source: Kohl and Stone, "Policy Gradient Reinforcement Learning for Fast Quadrupedal Locomotion," ICRA 2004.

A small parameterization made direct hardware evaluation feasible, but the evaluation count scales with dimension.

2.

Trajectory Objectives

CMU 10-703: Deep Reinforcement Learning

Policy Rollouts

Trajectory. A finite-horizon rollout records the states and actions generated by the policy and environment.

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T).$$

Trajectory return. The undiscounted reward accumulated over the rollout is

$$R(\tau) = \sum_{t=0}^T R(s_t, a_t).$$

Policy optimization ranks parameters by the returns of their induced trajectories.

Expected-Return Objective

Policy objective. Average trajectory return under the distribution induced by π_θ .

$$U(\theta) = \mathbb{E}_{\tau \sim P(\tau; \theta)}[R(\tau)] = \sum_{\tau} P(\tau; \theta) R(\tau).$$

Easy to sample

Roll out the current policy in the environment.

Hard to differentiate directly

The reward is observed after sampling a trajectory from a distribution that depends on θ .

The parameters affect expected return through the trajectory distribution, not through the realized reward function.

Trajectory Probability

$$P(\tau; \theta) = \rho(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t) \pi_{\theta}(a_t \mid s_t).$$

Environment terms

$$\rho(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t)$$

Independent of θ .

Policy terms

$$\prod_{t=0}^T \pi_{\theta}(a_t \mid s_t)$$

The only factors controlled by θ .

The policy gradient will not require differentiating the transition dynamics.

3.

Score-Function Gradients

CMU 10-703: Deep Reinforcement Learning

Log-Derivative Identity

$$\nabla_{\theta} \mathbb{E}_{x \sim P_{\theta}}[f(x)] = \nabla_{\theta} \sum_x P_{\theta}(x) f(x)$$

Begin with the definition of expectation under a parameterized distribution.

Log-Derivative Identity

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{x \sim P_{\theta}}[f(x)] &= \nabla_{\theta} \sum_x P_{\theta}(x) f(x) \\ &= \sum_x \nabla_{\theta} P_{\theta}(x) f(x)\end{aligned}$$

Linearity moves the gradient inside the sum.

Log-Derivative Identity

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{x \sim P_{\theta}}[f(x)] &= \nabla_{\theta} \sum_x P_{\theta}(x) f(x) \\ &= \sum_x \nabla_{\theta} P_{\theta}(x) f(x) \\ &= \sum_x P_{\theta}(x) \frac{\nabla_{\theta} P_{\theta}(x)}{P_{\theta}(x)} f(x)\end{aligned}$$

Multiplying and dividing by the probability exposes its logarithmic derivative.

Log-Derivative Identity

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{x \sim P_{\theta}}[f(x)] &= \nabla_{\theta} \sum_x P_{\theta}(x) f(x) \\ &= \sum_x \nabla_{\theta} P_{\theta}(x) f(x) \\ &= \sum_x P_{\theta}(x) \frac{\nabla_{\theta} P_{\theta}(x)}{P_{\theta}(x)} f(x) \\ &= \mathbb{E}_{x \sim P_{\theta}}[\nabla_{\theta} \log P_{\theta}(x) f(x)]\end{aligned}$$

The gradient is an expectation of a score-weighted function value.

Log-Derivative Identity

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{x \sim P_{\theta}}[f(x)] &= \nabla_{\theta} \sum_x P_{\theta}(x) f(x) \\&= \sum_x \nabla_{\theta} P_{\theta}(x) f(x) \\&= \sum_x P_{\theta}(x) \frac{\nabla_{\theta} P_{\theta}(x)}{P_{\theta}(x)} f(x) \\&= \mathbb{E}_{x \sim P_{\theta}} [\nabla_{\theta} \log P_{\theta}(x) f(x)] \\&\approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log P_{\theta}(x^{(i)}) f(x^{(i)}).\end{aligned}$$

Samples from P_{θ} provide an unbiased Monte Carlo gradient estimator.

Monte Carlo Policy Gradient

Assume P_θ is differentiable, easy to sample, and permits moving ∇_θ through the expectation. Substituting $x \leftrightarrow \tau$ and $f \leftrightarrow R$ gives

$$\nabla_\theta U(\theta) = \mathbb{E}_{\tau \sim P_\theta} [\nabla_\theta \log P_\theta(\tau) R(\tau)] \approx \frac{1}{N} \sum_{i=1}^N \nabla_\theta \log P_\theta(\tau^{(i)}) R(\tau^{(i)}).$$

Score

Measures how the log probability of a sampled trajectory changes with θ .

Weight

Multiplies the score by the observed trajectory return.

Convergence

The sample average converges as N grows.

The estimator needs sampled trajectories and policy probabilities, but no differentiable environment model.

Trajectory Score Factorization

$$\nabla_{\theta} \log P(\tau; \theta) = \nabla_{\theta} \log \left[\rho(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t) \pi_{\theta}(a_t \mid s_t) \right]$$

Expand the probability of the sampled trajectory.

Trajectory Score Factorization

$$\begin{aligned}\nabla_{\theta} \log P(\tau; \theta) &= \nabla_{\theta} \log \left[\rho(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t) \pi_{\theta}(a_t \mid s_t) \right] \\ &= \nabla_{\theta} \left[\log \rho(s_0) + \sum_{t=0}^T \log P(s_{t+1} \mid s_t, a_t) + \sum_{t=0}^T \log \pi_{\theta}(a_t \mid s_t) \right]\end{aligned}$$

The logarithm converts the product into sums of environment and policy terms.

Trajectory Score Factorization

$$\begin{aligned}\nabla_{\theta} \log P(\tau; \theta) &= \nabla_{\theta} \log \left[\rho(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t) \pi_{\theta}(a_t \mid s_t) \right] \\ &= \nabla_{\theta} \left[\log \rho(s_0) + \sum_{t=0}^T \log P(s_{t+1} \mid s_t, a_t) + \sum_{t=0}^T \log \pi_{\theta}(a_t \mid s_t) \right] \\ &= \nabla_{\theta} \sum_{t=0}^T \log \pi_{\theta}(a_t \mid s_t)\end{aligned}$$

Initial-state and dynamics terms vanish because they do not depend on θ .

Trajectory Score Factorization

$$\begin{aligned}\nabla_{\theta} \log P(\tau; \theta) &= \nabla_{\theta} \log \left[\rho(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t) \pi_{\theta}(a_t \mid s_t) \right] \\ &= \nabla_{\theta} \left[\log \rho(s_0) + \sum_{t=0}^T \log P(s_{t+1} \mid s_t, a_t) + \sum_{t=0}^T \log \pi_{\theta}(a_t \mid s_t) \right] \\ &= \nabla_{\theta} \sum_{t=0}^T \log \pi_{\theta}(a_t \mid s_t) \\ &= \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t).\end{aligned}$$

The trajectory score is the sum of per-action policy scores.

REINFORCE Estimator

REINFORCE. Use sampled trajectory returns to weight the log-probability gradients of every sampled action.

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) R(\tau^{(i)}).$$

Positive return

Increase the probability of sampled actions.

Negative return

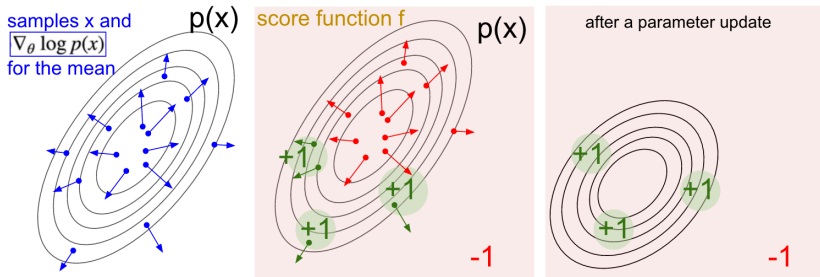
Decrease the probability of sampled actions.

Magnitude

Larger absolute returns produce larger updates.

REINFORCE is an unbiased score-function estimator, but its trajectory-level weights can have high variance.

Score-Weighted Policy Updates



Sample

Draw actions from the current distribution.

Weight

Scale score vectors by positive or negative reward.

Shift

Move probability mass toward positively weighted samples.

The score points toward parameters that make its sampled action more probable.

4.

Policy Class Gradients

CMU 10-703: Deep Reinforcement Learning

Univariate Gaussian Score

Assume a fixed variance and policy $\pi_{\theta}(a | s) = \mathcal{N}(a; \mu_{\theta}(s), \sigma^2)$.

$$\pi_{\theta}(a | s) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(a - \mu_{\theta}(s))^2}{2\sigma^2}\right).$$

$$\nabla_{\theta} \log \pi_{\theta}(a | s) = \frac{a - \mu_{\theta}(s)}{\sigma^2} \nabla_{\theta} \mu_{\theta}(s).$$

Actions above or below the mean push the mean in the corresponding direction, scaled by inverse variance.

Multivariate Gaussian Score

For fixed covariance Σ ,

$$\pi_{\theta}(a \mid s) = \frac{\exp\left[-\frac{1}{2}(a - \mu_{\theta}(s))^{\top} \Sigma^{-1}(a - \mu_{\theta}(s))\right]}{\sqrt{(2\pi)^k \det \Sigma}}.$$

$$\nabla_{\theta} \log \pi_{\theta}(a \mid s) = [\nabla_{\theta} \mu_{\theta}(s)]^{\top} \Sigma^{-1}(a - \mu_{\theta}(s)).$$

The covariance rescales action deviations before backpropagation through the policy mean.

Softmax Policies

Softmax policy. Normalize state-action logits into a categorical distribution.

$$\pi_{\theta}(a \mid s) = \frac{e^{h_{\theta}(s,a)}}{\sum_b e^{h_{\theta}(s,b)}}.$$

Linear logits

$$h_{\theta}(s, a) = \phi(s, a)^{\top} \theta$$

Neural logits

A network produces one score for each available discrete action.

Softmax couples the actions because raising one logit changes every normalized probability.

Softmax Score Derivation

$$\nabla_{\theta} \log \pi_{\theta}(a \mid s) = \nabla_{\theta} h_{\theta}(s, a) - \nabla_{\theta} \log \sum_b e^{h_{\theta}(s, b)}$$

Write log softmax as the selected logit minus a log normalizer.

Softmax Score Derivation

$$\begin{aligned}\nabla_{\theta} \log \pi_{\theta}(a \mid s) &= \nabla_{\theta} h_{\theta}(s, a) - \nabla_{\theta} \log \sum_b e^{h_{\theta}(s, b)} \\ &= \nabla_{\theta} h_{\theta}(s, a) - \frac{\sum_b e^{h_{\theta}(s, b)} \nabla_{\theta} h_{\theta}(s, b)}{\sum_b e^{h_{\theta}(s, b)}}\end{aligned}$$

Differentiate the log-sum-exp normalizer.

Softmax Score Derivation

$$\begin{aligned}\nabla_{\theta} \log \pi_{\theta}(a \mid s) &= \nabla_{\theta} h_{\theta}(s, a) - \nabla_{\theta} \log \sum_b e^{h_{\theta}(s, b)} \\&= \nabla_{\theta} h_{\theta}(s, a) - \frac{\sum_b e^{h_{\theta}(s, b)} \nabla_{\theta} h_{\theta}(s, b)}{\sum_b e^{h_{\theta}(s, b)}} \\&= \nabla_{\theta} h_{\theta}(s, a) - \sum_b \pi_{\theta}(b \mid s) \nabla_{\theta} h_{\theta}(s, b)\end{aligned}$$

The normalized exponential weights are the policy probabilities.

Softmax Score Derivation

$$\begin{aligned}\nabla_{\theta} \log \pi_{\theta}(a \mid s) &= \nabla_{\theta} h_{\theta}(s, a) - \nabla_{\theta} \log \sum_b e^{h_{\theta}(s, b)} \\&= \nabla_{\theta} h_{\theta}(s, a) - \frac{\sum_b e^{h_{\theta}(s, b)} \nabla_{\theta} h_{\theta}(s, b)}{\sum_b e^{h_{\theta}(s, b)}} \\&= \nabla_{\theta} h_{\theta}(s, a) - \sum_b \pi_{\theta}(b \mid s) \nabla_{\theta} h_{\theta}(s, b) \\&= \text{selected-action logit gradient} - \text{policy-average logit gradient}.\end{aligned}$$

The score raises the selected action relative to the current policy average.

5.

Temporal Structure

CMU 10-703: Deep Reinforcement Learning

Trajectory Return as a Noisy Weight

Basic REINFORCE uses the same trajectory return for every action score:

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) R(\tau^{(i)}).$$



Action a_t cannot affect rewards observed before time t , so past rewards add variance without useful credit.

Reward-to-Go Derivation

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \sum_{k=0}^T R(s_k^{(i)}, a_k^{(i)})$$

Expand the trajectory return into individual rewards.

Reward-to-Go Derivation

$$\begin{aligned}\hat{g} &= \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) \sum_{k=0}^T R(s_k^{(i)}, a_k^{(i)}) \\ &= \frac{1}{N} \sum_{i,t} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) \left[\sum_{k=0}^{t-1} R_k^{(i)} + \sum_{k=t}^T R_k^{(i)} \right]\end{aligned}$$

Separate rewards before and after the sampled action.

Reward-to-Go Derivation

$$\begin{aligned}\hat{g} &= \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) \sum_{k=0}^T R(s_k^{(i)}, a_k^{(i)}) \\ &= \frac{1}{N} \sum_{i,t} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) \left[\sum_{k=0}^{t-1} R_k^{(i)} + \sum_{k=t}^T R_k^{(i)} \right] \\ &\rightsquigarrow \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) \underbrace{\sum_{k=t}^T R_k^{(i)}}_{G_t^{(i)}}.\end{aligned}$$

Dropping past rewards preserves the expected gradient and yields the reward-to-go estimator.

Reward-to-Go Properties

Reward-to-go. The return from time t onward is

$$G_t^{(i)} = \sum_{k=t}^T R(s_k^{(i)}, a_k^{(i)}).$$

Unbiased

Removes terms whose expected score contribution is zero.

Lower variance

Excludes irrelevant past rewards.

Better credit

Each action is weighted only by later outcomes.

More stable

Cleaner weights reduce gradient noise.

Reward-to-go changes the weight across time steps while leaving the expected policy gradient unchanged.

REINFORCE Algorithm

Source: Williams (1992); Sutton et al. (1999).

1. Sample N trajectories $\{\tau^{(i)}\}_{i=1}^N$ using π_{θ} .

The current policy supplies the training trajectories.

REINFORCE Algorithm

Source: Williams (1992); Sutton et al. (1999).

1. Sample N trajectories $\{\tau^{(i)}\}_{i=1}^N$ using π_θ .
2. Compute every reward-to-go $G_t^{(i)} = \sum_{k=t}^T R(s_k^{(i)}, a_k^{(i)})$.

Each action receives a return containing only rewards it could affect.

REINFORCE Algorithm

Source: Williams (1992); Sutton et al. (1999).

1. Sample N trajectories $\{\tau^{(i)}\}_{i=1}^N$ using π_θ .
2. Compute every reward-to-go $G_t^{(i)} = \sum_{k=t}^T R(s_k^{(i)}, a_k^{(i)})$.
3. Estimate the policy gradient.

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) G_t^{(i)}.$$

Average score-weighted returns across trajectories and time steps.

REINFORCE Algorithm

Source: Williams (1992); Sutton et al. (1999).

1. Sample N trajectories $\{\tau^{(i)}\}_{i=1}^N$ using π_θ .
2. Compute every reward-to-go $G_t^{(i)} = \sum_{k=t}^T R(s_k^{(i)}, a_k^{(i)})$.
3. Estimate the policy gradient.

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) G_t^{(i)}.$$

Update $\theta \leftarrow \theta + \alpha \hat{g}$.

Repeat after updating the policy parameters.

6.

Variance Reduction

CMU 10-703: Deep Reinforcement Learning

High-Variance Gradient Estimates

The reward-to-go estimator is unbiased, but finite batches can still produce noisy directions.

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) G_t^{(i)}.$$

$$\text{Var}(\hat{g}) = \text{tr} \mathbb{E}[(\hat{g} - \mathbb{E}[\hat{g}])(\hat{g} - \mathbb{E}[\hat{g}])^{\top}] = \sum_{k=1}^n \mathbb{E}[(\hat{g}_k - \mathbb{E}[\hat{g}_k])^2].$$

Unbiasedness controls the mean estimate; variance determines how many trajectories are needed for a useful update.

Baseline Subtraction

Baseline. Subtract a quantity that does not depend on the sampled action, then use the centered return as the score weight.

$$\hat{g}_b = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) (G_t^{(i)} - b(s_t^{(i)})).$$

Center the signal

Compare a return with a reference level rather than with zero.

Preserve the mean

The expected score-weighted baseline is zero.

A well-chosen baseline reduces variation in the weights without changing the expected gradient.

Constant-Baseline Unbiasedness

$$\mathbb{E}_{\tau \sim P_{\theta}}[\nabla_{\theta} \log P_{\theta}(\tau) b] = \sum_{\tau} P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau) b$$

Write the expected baseline contribution under the trajectory distribution.

Constant-Baseline Unbiasedness

$$\begin{aligned}\mathbb{E}_{\tau \sim P_{\theta}}[\nabla_{\theta} \log P_{\theta}(\tau)b] &= \sum_{\tau} P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau)b \\ &= b \sum_{\tau} \nabla_{\theta} P_{\theta}(\tau)\end{aligned}$$

The score identity replaces probability times log-gradient by a probability gradient.

Constant-Baseline Unbiasedness

$$\begin{aligned}\mathbb{E}_{\tau \sim P_{\theta}}[\nabla_{\theta} \log P_{\theta}(\tau)b] &= \sum_{\tau} P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau)b \\ &= b \sum_{\tau} \nabla_{\theta} P_{\theta}(\tau) \\ &= b \nabla_{\theta} \sum_{\tau} P_{\theta}(\tau)\end{aligned}$$

Move the gradient outside the sum over all trajectories.

Constant-Baseline Unbiasedness

$$\begin{aligned}\mathbb{E}_{\tau \sim P_{\theta}}[\nabla_{\theta} \log P_{\theta}(\tau)b] &= \sum_{\tau} P_{\theta}(\tau) \nabla_{\theta} \log P_{\theta}(\tau)b \\ &= b \sum_{\tau} \nabla_{\theta} P_{\theta}(\tau) \\ &= b \nabla_{\theta} \sum_{\tau} P_{\theta}(\tau) \\ &= b \nabla_{\theta} 1 = 0.\end{aligned}$$

Probability normalization makes the expected baseline contribution exactly zero.

State-Dependent Baseline Unbiasedness

Conditioned on state s_t , the baseline $b(s_t)$ is fixed across actions.

$$\mathbb{E}_{a_t \sim \pi_\theta(\cdot | s_t)} [\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] = b(s_t) \sum_a \pi_\theta(a | s_t) \nabla_\theta \log \pi_\theta(a | s_t)$$

Condition on the state and average over the policy's action distribution.

State-Dependent Baseline Unbiasedness

Conditioned on state s_t , the baseline $b(s_t)$ is fixed across actions.

$$\begin{aligned}\mathbb{E}_{a_t \sim \pi_\theta(\cdot | s_t)}[\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] &= b(s_t) \sum_a \pi_\theta(a | s_t) \nabla_\theta \log \pi_\theta(a | s_t) \\ &= b(s_t) \nabla_\theta \sum_a \pi_\theta(a | s_t)\end{aligned}$$

The score sum is the gradient of the policy's normalization constraint.

State-Dependent Baseline Unbiasedness

Conditioned on state s_t , the baseline $b(s_t)$ is fixed across actions.

$$\begin{aligned}\mathbb{E}_{a_t \sim \pi_\theta(\cdot | s_t)}[\nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t)] &= b(s_t) \sum_a \pi_\theta(a | s_t) \nabla_\theta \log \pi_\theta(a | s_t) \\ &= b(s_t) \nabla_\theta \sum_a \pi_\theta(a | s_t) \\ &= b(s_t) \nabla_\theta 1 = 0.\end{aligned}$$

Any state-only baseline preserves the expected policy gradient.

Baseline Choices

$$\hat{g}_b = \frac{1}{N} \sum_{i,t} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) (G_t^{(i)} - b_t^{(i)}).$$

Constant

$$b = \mathbb{E}[R(\tau)]$$

One scalar centers all returns.

Time-dependent

$$b_t = \frac{1}{N} \sum_i G_t^{(i)}$$

One batch average for each time step.

State-dependent

$$b(s) = V^{\pi}(s)$$

A learned prediction for each state.

More informative baselines compare each action with a reference matched to its decision context.

Action Returns and Advantages

State	$V(s)$	$G(a_1)$	$G(a_2)$	Advantages
s_1	3000	3030	2970	+30, -30
s_2	-4000	-3970	-4030	+30, -30

Raw-return comparison

Actions in the inherently high-return state dominate the scale.

State-centered comparison

Both states express the same within-state preference between actions.

The policy should reward actions that outperform other choices from the same state, not actions that merely occur in good states.

Advantage Functions

Advantage. The action value relative to the policy's average value at the same state.

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} \mid s_t^{(i)}) A^\pi(s_t^{(i)}, a_t^{(i)}).$$

$$A^\pi(s, a) > 0$$

Make action a more likely at state s .

$$A^\pi(s, a) < 0$$

Make action a less likely at state s .

The advantage supplies a state-calibrated direction for the actor's update.

7.

Actor-Critic Methods

CMU 10-703: Deep Reinforcement Learning

Monte Carlo Value Fitting

The state-value baseline is unknown and must be estimated from sampled data.

$$\phi^* = \arg \min_{\phi} \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \left[V_{\phi}^{\pi}(s_t^{(i)}) - \underbrace{\sum_{k=t}^{T-1} R(s_k^{(i)}, a_k^{(i)})}_{G_t^{(i)}} \right]^2.$$

Collect

Sample complete trajectories.

Target

Compute empirical returns G_t .

Regress

Fit $V_{\phi}^{\pi}(s_t)$ by squared error.

Monte Carlo targets use complete returns: no bootstrapping, but potentially high target variance.

Temporal-Difference Value Fitting

Temporal-difference estimation replaces the full return with a one-step bootstrap target.

$$y_{\ell}(s, a, s', r) = r + \gamma V_{\phi_{\ell}}^{\pi}(s').$$

$$\phi_{\ell+1} = \arg \min_{\phi} \sum_{(s,a,s',r)} [r + \gamma V_{\phi_{\ell}}^{\pi}(s') - V_{\phi}^{\pi}(s)]^2.$$

Short target

Learns from individual transitions before an episode ends.

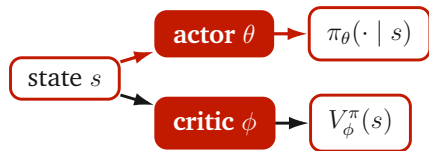
Bootstrap dependence

The target depends on the critic's current prediction at s' .

TD targets usually reduce variance and data latency, while introducing approximation and bootstrap error.

Actor-Critic Architectures

Separate networks



Simple optimization, but no shared features.

Shared representation



More feature reuse, but coupled optimization.

Actor and critic require distinct outputs; their feature extractors may be separate or shared.

Actor-Critic Algorithm

Source: Konda and Tsitsiklis (1999); Sutton and Barto (1998).

1. Sample trajectories with the actor π_θ .

The actor determines the state-action data available to both networks.

Actor-Critic Algorithm

Source: Konda and Tsitsiklis (1999); Sutton and Barto (1998).

1. Sample trajectories with the actor π_θ .
2. Fit the critic V_ϕ^π using Monte Carlo or TD targets.

The critic learns the state-dependent baseline used by the actor.

Actor-Critic Algorithm

Source: Konda and Tsitsiklis (1999); Sutton and Barto (1998).

1. Sample trajectories with the actor π_θ .
2. Fit the critic V_ϕ^π using Monte Carlo or TD targets.
3. Estimate action advantages.

$$\hat{A}_t^{(i)} = G_t^{(i)} - V_\phi^\pi(s_t^{(i)}).$$

Subtract the critic's prediction from each sampled return.

Actor-Critic Algorithm

Source: Konda and Tsitsiklis (1999); Sutton and Barto (1998).

1. Sample trajectories with the actor π_θ .
2. Fit the critic V_ϕ^π using Monte Carlo or TD targets.
3. Estimate action advantages.

$$\hat{A}_t^{(i)} = G_t^{(i)} - V_\phi^\pi(s_t^{(i)}).$$

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) \hat{A}_t^{(i)}.$$

Use estimated advantages to weight policy scores.

Actor-Critic Algorithm

Source: Konda and Tsitsiklis (1999); Sutton and Barto (1998).

1. Sample trajectories with the actor π_θ .
2. Fit the critic V_ϕ^π using Monte Carlo or TD targets.
3. Estimate action advantages.

$$\hat{A}_t^{(i)} = G_t^{(i)} - V_\phi^\pi(s_t^{(i)}).$$

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) \hat{A}_t^{(i)}.$$

Update $\theta \leftarrow \theta + \alpha \hat{g}$, then collect data with the improved actor.

Policy Gradient Summary



Model-free

Dynamics appear in samples but not in the differentiated score.

Unbiased

REINFORCE estimates the gradient of expected return.

Variance-aware

Reward-to-go and baselines remove irrelevant variation.

Actor-critic

A learned value baseline converts returns into advantages.

Policy gradients supply the actor update; temporal structure and a critic make that update statistically useful.

Readings

1. Williams, “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning,” 1992.
2. Sutton, McAllester, Singh, and Mansour, “Policy Gradient Methods for Reinforcement Learning with Function Approximation,” 1999.
3. Konda and Tsitsiklis, “Actor-Critic Algorithms,” 1999.
4. Karpathy, “Deep Reinforcement Learning: Pong from Pixels,” 2016.
5. Kohl and Stone, “Policy Gradient Reinforcement Learning for Fast Quadrupedal Locomotion,” 2004.

The original papers establish the estimators; the implementation readings show how those estimators behave in practice.